

NodeMCU:

Primeros ejemplos.

Escuela de Ingeniería Industrial de Toledo

Autor:

Juan Moreno García



NodeMCU: Primeros ejemplos

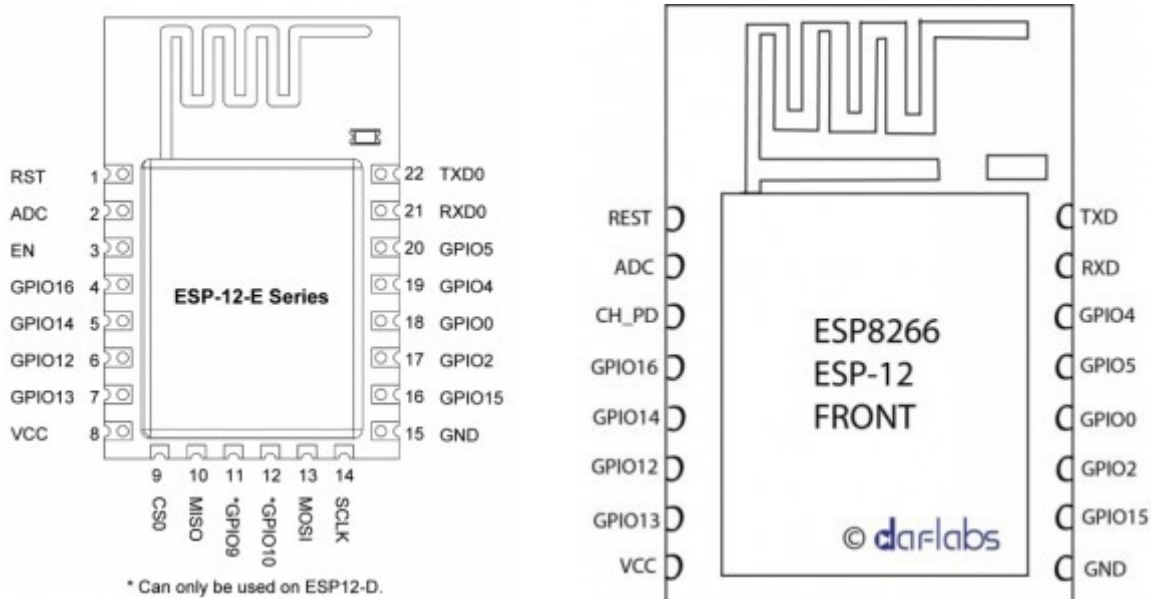
En este documento se describe brevemente la arquitectura del *NodeMCU*, la instalación en *Windows* y se muestran algunos ejemplos sencillos que hace uso del led y el botón integrado en el mismo. Es una primera aproximación al *NodeMCU*. A continuación se muestra el índice del documento.

Índice:	Página
1.- ESP-NN y NodeMCU.	4
2.- Un ejemplo más avanzado.	10

1.- ESP-NN y NodeMCU.

Las primeras placas que se lanzaron fueron las *ESP-NN*, por ejemplo *ESP01* o *ESP12*, que permiten la conexión *Wifi*. Su montaje es relativamente complejo siendo necesario soldar sus componentes al convertor de alimentación. Para más información ir hay algunas webs que facilitan su montaje, a día de hoy se encuentra la web <https://www.vispo.org/2015/07/21/puesta-en-marcha-del-soc-wifi-esp8266-esp-12-en-una-protoboard/> que cuenta el proceso con bastante detalle. Esta opción es interesante si se desea montar una PCB propia.

La versión 12E, versión mejorada (Enhanced en inglés), dispone de 6 pines más, con los CS0, MISO, MOSI y SCLK dedicados a la comunicación SP. Los nuevos pines se muestran de la siguiente figura:



Fuente: <https://www.prometec.net/nodemcu-arduino-ide/>

NodeMCU es un proyecto de código abierto que nace como una placa de desarrollo que cuenta con el microcontrolador *ESP8266*. Facilita mucho su programación dado que contiene toda la electrónica necesaria en la placa. Por otro lado, permite su conexión directamente al PC mediante un cable USB. Lo que permite su alimentación, poder copiar los programas en su memoria flash (flashear) y la comunicación con el PC a través de una conexión RS-232 lógica.

Está pensado para programarse en *LUA*, pero posteriormente se desarrollaron opciones para su programación mediante el *IDE* de *Arduino*. Además, se puede cargar cualquier *firmware*, lo que posibilita la utilización de otros lenguajes, como por ejemplo, *MicroPython*, un *Python* reducido. En nuestro caso utilizaremos el lenguaje de *Arduino*.

El uso del plugin del *ESP8266* para programar el *NodeMCU* en *Arduino* ofrece las siguientes ventajas:

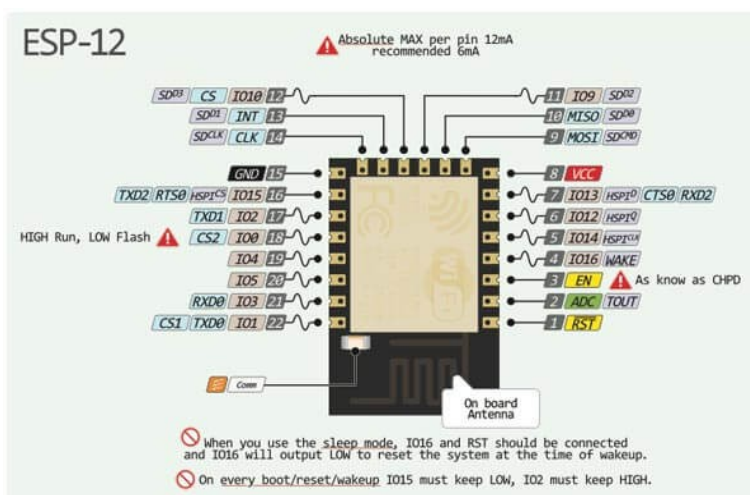
1. La posibilidad de utilizar el entorno IDE de *Arduino*, conocido por muchos programadores
2. El lenguaje de programación es el mismo que el de *Arduino*, con lo que se puede programar con los conceptos aprendidos ya sea en C o en el mismo *Arduino*. Esto permite programar el microprocesador *ESP8266* como un *Arduino*.

3. La programación de la Wifi no utiliza una librería que similar a la librería Wifi de Arduino, pudiendo reusar muchos de los programas desarrollados.

NodeMCU tiene la ventaja respecto a otros módulos con un ESP2866 de que los pines están en el exterior, pudiendo hacer montajes en una placa de pruebas (protoboard en inglés), y cuenta con un conector mini USB para comunicarse con el PC y para programarlo.

ESP-12

NodeMCU contiene un ESP12E, módulo que consta de un microprocesador, una antena PCB, un LED conectado al GPIO2 y una memoria flash de 4 MB (16 Mbit). La siguiente figura muestra su esquema.



Fuente: <https://programarfácil.com/esp8266/como-programar-nodemcu-ide-arduino/>

ESP12 se alimenta con 3.3 V, pero gracias a un regulador de tensión que incorpora el *NodeMCU* se puede alimentar mediante el cable *USB* con 5V. El regulador tiene un consumo residual de 8mA, lo que puede ser mucho en un proyecto que necesite bajo consumo. Pero es ideal para prototipado y aprendizaje por sus prestaciones y bajo precio, además se puede utilizar en proyectos donde el consumo residual pueda ser superior al límite indicado anteriormente.

La placa tiene 2 botones (*RST* y *FLASH*), uno conectado al pin de *RESET* y otro al *GPIO0* que activa el modo de carga de *firmware*. No es necesario utilizar estos botones cuando se programa desde el *IDE* de Arduino.

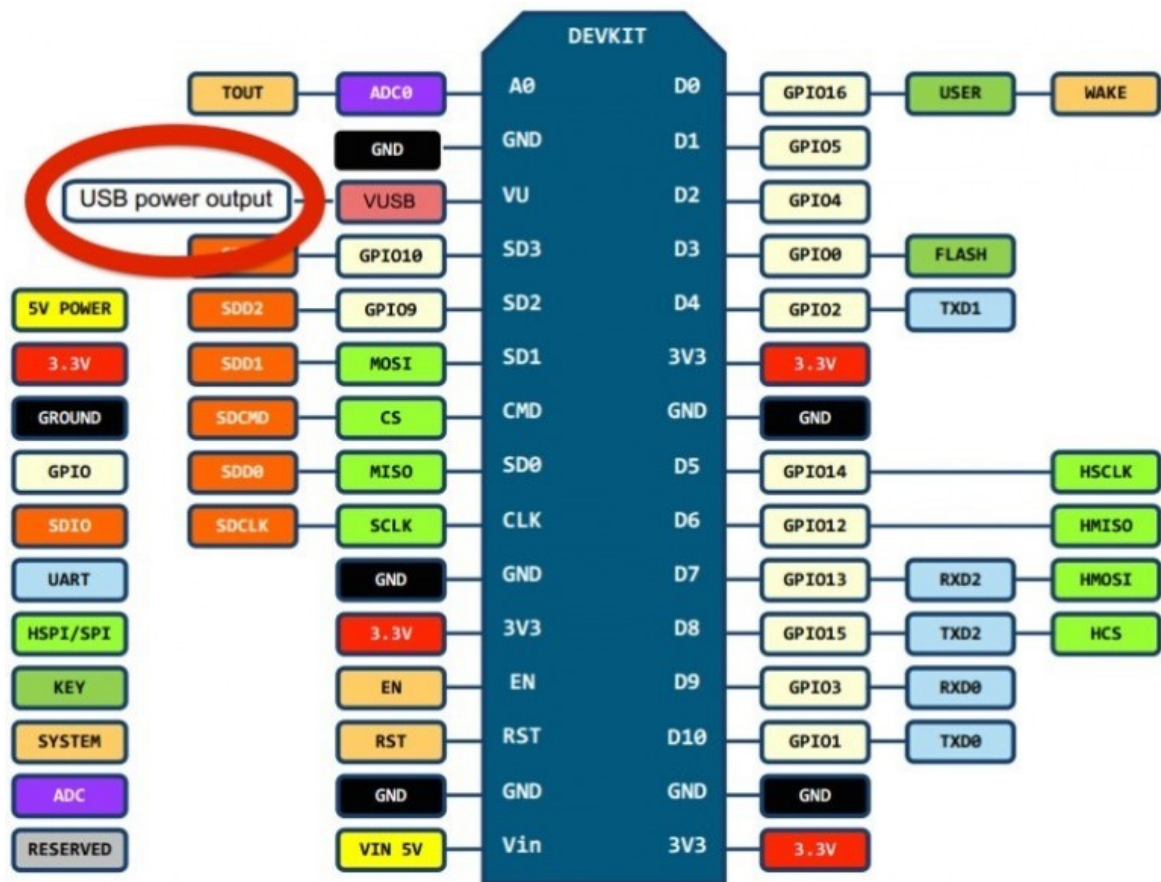
El *NodeMCU* cuenta con 11 pines de entrada salida y el puerto de entrada analógica. Los pines *GPIO* (*General Purpose Input/Output, Entrada/Salida de Propósito General*) del 6 al 11 están también conectados, y el *ESP12E* los utiliza para conectar la memoria flash. Si se utilizan es posible que el módulo se reinicie de forma inesperada.

La electrónica del *NodeMCU* simula la secuencia de pulsaciones para entrar en el modo de carga de *firmware*. Los pines *GPIO* 1 y 3 corresponden con el *Rx* y el *Tx* del puerto serie. Estos pines están protegidos con resistencias de 470 Ohm.

NodeMCU también facilita el uso del puerto analógico. El ESP8266 tiene un rango de entrada en este puerto de 0 a 1V pero gracias a un divisor de tensión *NodeMCU* lo extiende de 0 a 3.3V.

Pines del NodeMCU

La siguiente figura muestra los pines del *NodeMCU*, así como su conexión con los pines del *ESP-12*.



Fuente: <https://www.prometec.net/nodemcu-arduino-ide/>

El nombre de los pines del *NodeMCU* tiene una nomenclatura diferente de utilizada en el *ESP8266*. Tanto el número que identifica al pin, como la forma completa de identificarlo es diferente. Los pines del *NodeMCU* comienzan por la letra D, por ejemplo, los pines D0, D1, y D2 son los GPIO16, 5 y 4 del *ESP8266*. El framework para *Arduino* permite utilizar ambas para referirse a los pines de entrada salida, pudiendo utilizar las dos nomenclaturas indistintamente. Por ejemplo, estas dos sentencias son equivalentes (la columna izquierda utiliza la notación del *NodeMCU*). Nosotros utilizaremos los pines del *ESP8266*.

```
pinMode(D1, OUTPUT);
digitalWrite(D1, HIGH);
```

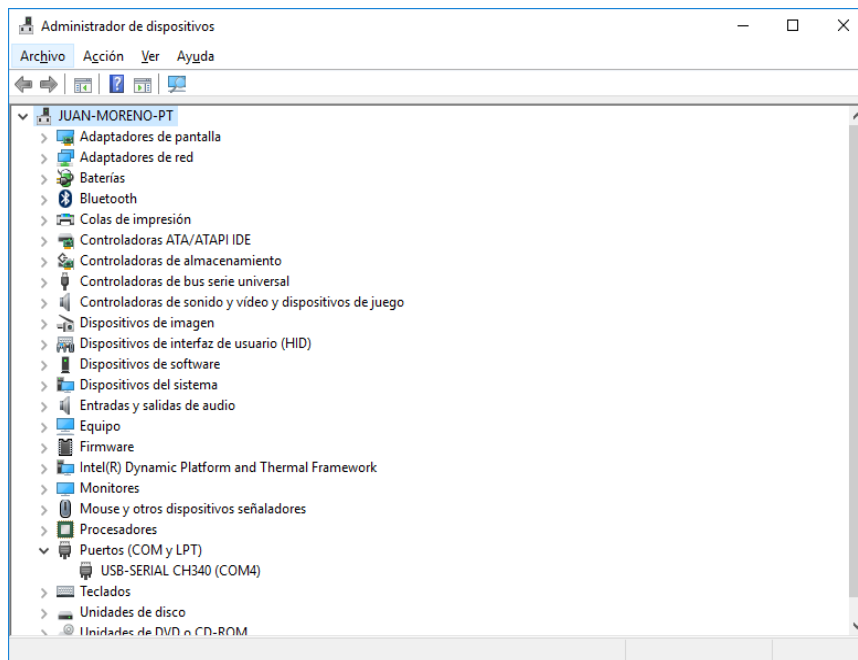
```
pinMode(5, OUTPUT);
digitalWrite(5, HIGH);
```

En la figura anterior se muestra cómo el pin *D1* del *NodeMCU* está conectado al pin 5 del *ESP12E*.

El *NodeMCU V3* (versión V1.0) es una versión creada por LoLin y que aporta algunas ventajas sobre la versión V1.0. LoLin ha utilizado uno de los pines reservados para la alimentación USB de salida y otro para una conexión GND adicional, además defienden que el puerto USB es más robusto.

Conectando el NodeMCU al PC

Las placas *NodeMCU* incorporan un conversor *USB* a serie. Solamente es necesario conectar el puerto *USB* de la placa a uno de los puertos del ordenador con un cable *USB*. *Windows* normalmente detectará el hardware e instala los drivers automáticamente. Existen diferentes fabricantes de estas tarjetas que pueden usar distintos conversores. Los más utilizados son el CH340 del fabricante chino WCH y el CP1202 de Silabs. La siguiente figura muestra cómo el *Administrador de dispositivos* de *Windows* muestra la conexión del *NodeMCU* utilizando el puerto *COM4*. Desde el *Administrador de Windows* se puede ver al puerto al que está conectado.



Conexión del *NodeMCU* al PC utilizando el puerto lógico *COM4*.

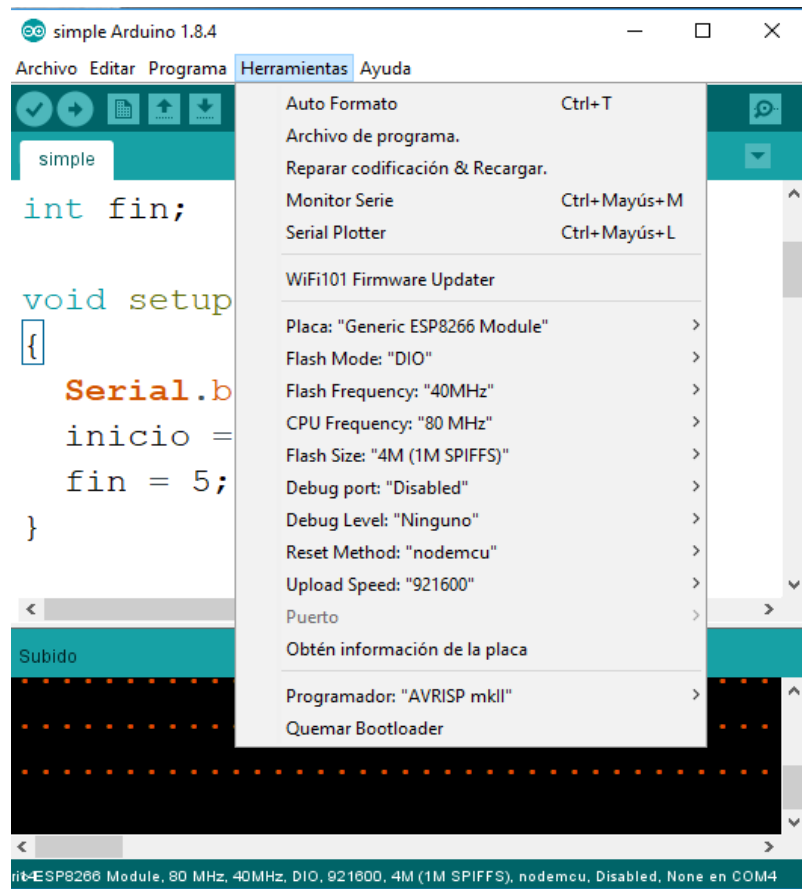
En principio, estos dos chips funcionan correctamente con el máximo *bitrate* que soporta el *IDE* de *Arduino* (921kbps). Es importante utilizar la máxima velocidad posible para que la carga del programa en la memoria flash del *ESP8266* se realice lo más rápidamente posible. Si no funciona correctamente a la máxima velocidad hay que probar el resto de velocidades hasta dar con la que funciona correctamente en el sistema y que sea la más alta posible.

El vídeo que se encuentra en la dirección <https://youtu.be/0g7sazWXfEI> muestra cómo configurar y realizar un primer programa utilizando el *ESP12* (integrado en *NodeMCU*) con el *IDE* de *Arduino*. Es muy interesante la visualización del mismo para comprender el proceso de instalación del *NodeMCU* en *Windows*. Seguir los pasos y explicaciones indicadas en el mismo para una correcta instalación.

Las opciones de configuración de la placa es una importante acción dentro de todo este proceso. La placa debe ser *Generic ESP8266 Module*. Al seleccionar la placa se carga automáticamente una configuración. Es importante cambiar las siguientes opciones para un funcionamiento más eficiente:

- *Flash Size* a *4M (1M SPIFFS)*. Esto si es para la versión *NodeMCU V3*.
- *Reset Method* a *nodemcu*. Esto permite el flasheo de la memoria automáticamente (sin tener que pulsar el botón *FLASH*).
- *Upload Speed* a *921600* si es posible. A más velocidad de carga más rápido se realiza la carga de los programas. El monitor serie puede funcionar a diferente velocidad. Si el *IDE* de

Arduino no puede programar la placa se debe bajar este campo hasta dar con la correcta, a 115200 bps suele funcionar correctamente.



Una posible configuración de opciones.

Algunos ejemplos iniciales

Dentro de *Arduino* y *NodemMCU* el equivalente al clásico “*Hola Mundo*” es el uso del led integrado en ambos. El LED está conectado al *GPIO2*. El siguiente programa hace que el *LED* se encienda durante un segundo y se apague durante 2,5 segundos.

```
#define LED_BUILTIN 2

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // LED integrado conectado al pin 2.
}

void loop() {
  digitalWrite(LED_BUILTIN, LOW); // LED encendido si está a nivel bajo
  delay(1000); // Esperar un segundo
  digitalWrite(LED_BUILTIN, HIGH); // Apagar el LED a nivel alto
  delay(2500); // Esperar 2,5 segundos
}
```

Primeramente se ha de configurar el modo del pin (*INPUT* o *OUTPUT*) mediante la orden *pinMode()*, que tiene dos parámetros, el número de pin y el modo de funcionamiento en el *setup()*. A continuación se debe indicar las órdenes que provocan el encendido o apagado del LED y el retardo temporal. Éstas son *digitalWrite()* y *delay()* respectivamente. *digitalWrite()* tiene los parámetros número de puerto y valor asignado (*LOW* o *HIGH*).

El segundo ejemplo configurará el botón *FLASH* del *NodeMCU* como un pulsador que permite encender y apagar el LED. Este botón se utiliza para poner en modo *carga del firmware*, pero con el *IDE* de *Arduino* no es necesario para la carga de los programas en *NodeMCU* siempre que se active la opción *Reset Method* a *nodemcu*. Una vez cargado el programa se puede utilizar como un pulsador normal. El GPIO del botón *FLASH* es el *GPIO0*.

El siguiente programa enciende el *LED* cuando se pulsa el botón *FLASH*.

```
#define LED_BUILTIN 2
#define BUTTON_BUILTIN 0

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // LED integrado conectado a pin 2.
  pinMode(BUTTON_BUILTIN, INPUT);
}

void loop() {
  // Leer el estado del botón. Pulsado está a nivel bajo
  int estado = digitalRead(BUTTON_BUILTIN);
  digitalWrite(LED_BUILTIN, estado); // Configurar LED como estado botón
}
```

La función *digitalRead()* se utiliza para realizar una lectura del botón *FLASH* (pin 0) que previamente ha sido configurado como *INPUT* utilizando la orden *pinMode(BUTTON_BUILTIN, INPUT)*. *digitalRead()* devuelve *LOW* o *HIGH* según esté pulsado o sin pulsar respectivamente. Destacar que tanto para el LED como para el botón, *LOW* significa que está activa (encendido o pulsado respectivamente).

El último ejemplo mostrará el uso de un contador que lleva el número de veces que se ha pulsado el botón *FLASH*. Para llevar la cuenta se utiliza la variable global *count* que se inicializa a 0 en la función *setup()*. Además, se utiliza el monitor serie para mostrar su valor cada vez que se pulsa el mismo. Se ha implementado la función *pulsado()* que devuelve un valor *booleano* según esté pulsado o no el botón (*true* o *false* respectivamente). Esta función cuenta con un *delay()* para tratar con el problema del rebote con los botones, esto es una solución pobre que nos valdrá en este punto, posteriormente se mostrará cómo solucionarlo de una mejor forma.

```
#define LED_BUILTIN 2
#define BUTTON_BUILTIN 0
// Variable global para contar pulsaciones
byte count;
void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT); // El LED está conectado al pin 2
  pinMode(BUTTON_BUILTIN, INPUT);
  count = 0; // Se inicializa a 0 pulsaciones
}
void loop() {
  // Leer el estado del botón, cuando está pulsado se pone a nivel bajo
  if (pulsado(BUTTON_BUILTIN)) {
    // Configurar el LED de acuerdo al estado del botón
    digitalWrite(LED_BUILTIN, LOW);
    count++; // Un nueva pulsación más
    Serial.println(count);
  } else {
    digitalWrite(LED_BUILTIN, HIGH);
  }
}
```

```

bool pulsado(byte pin) {
    int estado = digitalRead(pin);
    delay(250); // retardo para el rebote
    if (estado==LOW) { // Botón pulsado
        return true;
    } else {
        return false;
    }
}

```

2.- Un ejemplo más avanzado.

En esta sección se presentará un ejemplo un poco más complejo que muestra cómo leer información desde el monitor serie del IDE de Arduino, procesarla y mostrar el resultado en el *Monitor Serie*. Más concretamente, lee un valor entero y uno real, y, mediante dos invocaciones a la función llamada *a_cadena(float f, int n)*, genera una cadena que repite *n* veces el *float* leído separado por espacios en blanco. Por ejemplo, si *n=4* y *f=2.3* el resultado sería el *String* "2.30 2.30 2.30 2.30".

Para poder realizar este programa es necesaria la función *leerLinea()* que lee una cadena desde el puerto RS232 (Monitor Serie en este punto, ya se detallará esto más adelante). Esta función ha sido implementada por mí y será utilizada en muchos programas a partir de ahora. A partir de este momento se debe controlar su uso, más adelante se explicará su funcionamiento en detalle.

Para generar la cadena resultado se ha implementado la función *a_cadena(float f, int n)* que devuelve un *String* con la cadena resultado. Como puede verse, tiene como parámetros el número de veces que debe repetir el valor *float*. En este punto se debe entender que tiene dos parámetros, uno entero y otro real, y que devuelve la cadena resultado. Su cuerpo cuenta con un bucle *for* que va acumulando el resultado en el *String* *r*.

Finalmente el programa, la función *loop()*, lee el valor entero y posteriormente lee el valor real, invoca a la cadena y finalmente invoca a la función *a_cadena(float f, int n)*.

El código es el siguiente:

```

String a_cadena(float f, int n) {
    int i; // Declaración de variable entera
    String r=""; // Declaración de variable String
    for(i=0; i<n; i++) { // Bucle que acumula en r el resultado
        r += " " + String(f);
    }
    return r; // Devolución del resultado
}

void setup() {
    Serial.begin(115200); // Inicialización de puerto serie
}

void loop() {
    String r = ""; // Variable lee la entrada y obtener el resultado
    int n; // Contendrá el numero de veces a repetir el valor
    float f; // Valor real a introducir en el String resultado
}

```

```

    // Lectura del entero n
    Serial.println("Dame el número de veces"); // Muestra por el puerto
serie
    // recibe String introducido por puerto serie en r hasta '\n'
    r = leerLinea();
    n = r.toInt(); // Convierte String r a int
    // Lectura del float f
    Serial.println("Dame el numero float"); // Muestra por el puerto
serie
    // recibe String introducido por puerto serie en r hasta '\n'
    r = leerLinea();
    f = r.toFloat(); // Convierte String r a float
    // Invocación a la función a_cadena()
    r = a_cadena(f, n);
    Serial.println("La cadena resultado es: " + r);
}

String leerLinea() {
    // Espera hasta que haya algo que leer
    while(Serial.available() < 1) {
        delay(1);
    }
    return Serial.readStringUntil('\n'); // Recibe info hasta '\n'
}

```